30. Troubleshooting Method: REDUCE and Bisect

30. Troubleshooting Method: REDUCE and Bisect

Phase: Phase 10 — Observability and troubleshooting
Estimated time: 1–2 days
Slug: `30-troubleshooting`

Learning objectives

- Apply a systematic troubleshooting loop
- Capture evidence before rebooting
- Bisect recent changes
- Write short incident notes

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

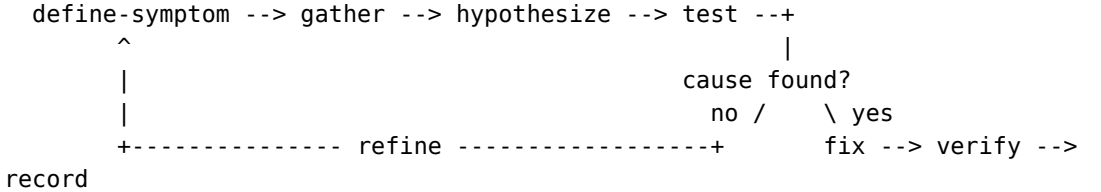
Why this matters

Most time lost in operations is not spent fixing; it is spent wandering. Engineers change random settings, restart services “to see”, and destroy the evidence that would have shown the cause. A repeatable **troubleshooting method** turns panic into a loop you can run in any incident, on any system, including ones you have never seen.

Core concepts

The method in six steps:

Step	Name	Question answered
1	define-symptom	What exactly is wrong, for whom, since when?
2	gather	What do logs, metrics, and state say right now?
3	hypothesize	What are the 2-3 most likely causes?
4	test	Which cheap, safe test separates the hypotheses?
5	fix	Apply one change, minimal and reversible
6	verify and record	Is the symptom gone? What did we learn?



The first step is always **define-symptom**. “The website is down” becomes: “`GET /health` returns 502 from the office network since 09:40; staging is fine; the last deploy was at 09:35.” A precise symptom prunes the search tree.

A second habit is to **work the layers**, bottom to top or top to bottom, never randomly:

```
app      : is the process running? errors in its log?
service  : systemctl status, ports listening (ss -ltnp)
network  : can I reach the host? DNS? firewall? (ping, curl, dig)
os       : disk full? memory? load? (df, free, uptime)
hardware : dmesg errors, SMART, NIC link
```

Worked examples

Symptom: “the app returns 502.”

```
# 1 define: who, what, when
curl -sS -o /dev/null -w '%{http_code}\n' http://127.0.0.1:8080/health
# 2 gather
systemctl status myapp --no-pager
journalctl -u myapp -n 50 --no-pager
ss -ltnp | grep 8080
df -h /; free -h
# 3 hypothesize: process dead? port taken? disk full? bad config?
# 4 test cheaply
ss -ltnp | grep -c ':8080'      # 0 means nothing listening
journalctl -u myapp -p err -n 5
# 5 fix one thing
sudo systemctl restart myapp
# 6 verify
curl -sS -o /dev/null -w '%{http_code}\n' http://127.0.0.1:8080/health
```

Symptom: “cannot SSH to the host.”

```
ping -c2 HOST             # network up?
nc -vz HOST 22            # port reachable?
ssh -v user@HOST          # which stage fails: connect, key, auth?
```

Bisect a change: if the system worked yesterday, ask “what changed?”

```
ls -lt /etc | head
journalctl --since yesterday | grep -i -E 'upgrade|install'
last -x | head
```

Compare with a known-good machine:

```
diff <(ssh good 'sysctl -a 2>/dev/null' | sort) <(sysctl -a 2>/dev/null | sort)
```

Write notes as you go:

```
10:02 symptom: 502 on /health since 09:40
10:05 gather: myapp inactive (dead), journal: "Permission denied: /var/lib/app"
10:08 hypothesis: dir ownership changed by deploy
```

```
10:09 test: ls -ld /var/lib/app -> root:root
10:10 fix: chown app:app /var/lib/app; restart
10:11 verify: 200 OK
```

Mini-lab

1. Break a service on purpose: create a systemd unit or a script that exits 1, or stop `cron` in a throwaway VM.
2. Write the symptom in one sentence before touching anything.
3. List three hypotheses and one test for each. Run the tests in order of cheapest first.
4. Apply a single fix, verify, and write a three-line incident note.
5. Repeat with a different fault: fill a small tmpfs (`mount -t tmpfs -o size=1m none /mnt/t`) and see how the app fails.

Decision guide / failure modes

Situation	Do this
Symptom vague	Reproduce it, capture exact error text and time
Many possible causes	Test the cheapest or most likely first
Problem started after a change	Roll back or diff the change first
Cannot reproduce	Add logging or metrics, wait for next occurrence
Production at risk	Mitigate first (failover, rollback), diagnose after
Stuck for 30 min	Explain the problem aloud to someone; recheck assumptions

Failure modes:

- **Changing many things at once**: you will not know which one fixed it.
- **Restart first**: a restart clears memory, locks, and logs that held the clue. Capture state (`ss`, `ps`, `journalctl`) before restarting.
- **Trusting assumptions**: “DNS is fine” must be tested with `dig`, not remembered.
- **Confirmation bias**: look for evidence that would disprove your favorite hypothesis.
- **No record**: the same incident returns in a month. Write down cause, fix, and detection.
- **Skipping verification**: “should work now” is not a test. Run the original failing action again.

Common mistakes

- Changing five things at once
- Reboot as first step always

- No notes after 3am fixes

Exercises

Work in `exercises/30-troubleshooting/`. Read that folder's `README.md` fully before coding.

```
cd exercises/30-troubleshooting
chmod +x test.sh exercise.sh
./test.sh
```

What you will build

Implement `first_troubleshooting_step` in `exercise.sh` so it prints the name of the very first step in the method from this lesson. Use the exact hyphenated token from the table (lowercase, with a hyphen, no spaces). Run `./test.sh` until green. Then write the six steps from memory in your notes and apply them to one real problem you have had recently. Keep the function body to one line; the goal is recall of the lesson vocabulary, not cleverness. Finish by re-reading the Decision guide table and noting one failure mode you could have caused yourself.

Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

Do not open `solutions/30-troubleshooting/` until you have an honest failing attempt.

Self-check

Try to answer without scrolling up:

1. Why capture evidence first?
2. What is bisection?
3. Name three evidence sources

Further reading

- https://sre.google/sre-book/